

用 Typst 写一本小书

一个面向文档工程的简短示例

Desc inno · 2026

一. 为什么是 Typst

长文档真正困难的地方，通常不是“把文字放到页面上”，而是让结构、内容与版式彼此保持独立。Typst 将文档写作和排版规则放在同一个可编程系统中，使得一份文档既可以保持良好的语义结构，又能够拥有稳定的视觉呈现。

定义 Definition

文档工程可以理解为：使用结构化数据、程序化规则和可复现的构建流程来生产文档，而不是手工调整每一页的视觉结果。

对于一本书而言，这种区别尤其明显。书稿可能不断增加章节、插图、脚注和代码示例；如果排版依赖人工调整，那么每次修改都可能产生新的连锁问题。

1.1 从“写页面”转向“写结构”

一个好的文档首先应该有清晰的结构：

- 书籍
 - 章节
 - 小节
 - 段落
- 图表与代码作为内容的一部分
- 脚注承担必要但不宜打断正文的说明
- 边注承担轻量的旁支信息

例如，`#heading` 表达的是“这里开始一个新的结构单元”，而不是“这里需要使用 15pt 的字体”。

原则 Principle

内容决定结构，模板决定外观。

这里使用模板提供的外侧边注。边注适合放置术语、补充说明或与正文存在弱关联的信息，不宜承载理解正文所必需的内容。

在正文中尽量使用语义化元素，而不是重复写字号、间距和颜色。这样修改模板时，正文通常无需改动。

1.2 一个最小的 Typst 文档

下面是一个足够小、但已经具有实际意义的 Typst 文档：

```
#set document(title: "Hello Typst")
```

```
≡ Hello, Typst
```

Typst 可以直接处理中文、公式、代码和交叉引用。

这里的 `#set`、标题和普通段落分别承担不同职责。模板则可以进一步统一字体、页边距、页眉以及标题样式。

提示 Tip

行内代码也会自动采用模板配置的等宽字体，例如 `#show`、`heading` 和 `counter(page)`。

二. 一份可维护的技术文档

2.1 数据与表示

假设我们需要比较几种常见文档格式。与其在正文中手工排列数据，不如让表格本身成为结构化内容。

格式	结构化	可编程	典型用途
Markdown	是	有限	README、笔记
LaTeX	是	强	学术论文、复杂出版
Typst	是	强	论文、报告、书籍

表格中的数据仍然属于正文内容，因此修改数据并不会改变文档的整体排版逻辑。

2.2 数学表达

技术文档通常还需要数学公式。例如，一个简单的复合增长模型可以写成：

$$V_t = V_0(1 + r)^t$$

其中 V_0 是初始值， r 是增长率， t 是时间， V_t 则表示经过 t 个周期之后的值。

示例 Example

当 $V_0 = 100$ 、 $r = 5\%$ 、 $t = 3$ 时，

$$V_3 = 100(1 + 0.05)^3 \approx 115.76$$

公式和普通正文一样参与页面布局，而不需要手工计算其位置。

2.3 列表与任务分解

在实际写作中，列表往往比连续的说明文字更适合表达操作步骤：

1. 先确定文档的语义结构。
2. 再定义统一的版式规则。
3. 最后处理特殊内容，例如公式、表格和代码。

也可以使用无序列表记录检查项：

- 标题层级是否合理
- 代码示例是否可以运行
- 脚注是否真正有必要
- 表格是否仍然易读
- 页眉信息是否足以定位当前章节

警告 **Warning**

不要为了展示排版能力而增加没有信息价值的装饰元素。对于技术文档，视觉层级应该服务于阅读，而不是反过来。

三. 代码、引用与可复现性

3.1 一个真实的函数

Typst 的一个重要特点是：样式本身可以被程序化。

例如，可以定义一个简单的函数，把重复结构封装起来：

```
#let badge(label) = box(  
  inset: (x: 5pt, y: 2pt),  
  radius: 2pt,  
  fill: luma(240),  
  [#label],  
)
```

这是一个 `#badge[Typst]` 示例。

模板中的 `callout` 也采用了类似的思想：不同类型的提示共享一个底层实现，只通过配置决定标题、背景和强调色。

参考资料 Reference

如果文档需要大量自定义组件，建议优先将它们封装为函数或 `show rule`，而不是在正文中复制格式代码。

3.2 脚注应该放在哪里

有些信息值得保留，但没有必要让它成为正文的一部分。例如，对一个术语进行简短解释时，脚注通常比括号中的长句更自然。Typst 的脚注会自动编号并放置到页面底部。¹

模板还会按一级标题重置脚注计数，因此每个章节可以拥有独立的脚注编号体系。

¹脚注适合补充说明，但不应成为正文论证的主要载体。

3.3 适度使用提示

并非每一个值得强调的句子都需要 `callout`。一个比较自然的技术文档通常只在语义发生变化时使用它们：

证据 Evidence

观察结果。模板能够在不修改正文结构的情况下统一控制字体、段落和标题样式。

推断 Inference

由此可以推断。当排版规则集中在模板中时，长文档的维护成本通常会低于逐页调整。

剩余内容则继续使用普通段落。这样 `callout` 才具有真正的视觉信号意义，而不会因为过度使用而失去层级。

四. 从模板到成书

4.1 章节组织

当文档规模扩大后，最自然的组织方式不是把所有内容塞进一个文件，而是让入口文件负责元数据和整体结构，再将章节拆分到独立文件。

例如：

```
book/
├── demo.typ
├── template.typ
├── chapters/
│   ├── introduction.typ
│   ├── typesetting.typ
│   └── conclusion.typ
└── figures/
    └── architecture.svg
```

入口文件可以使用模板提供的 `chapters` 参数：

```
#show: book.with(
  title: "用 Typst 写一本小书",
  author: "Desc inno",
  chapters: (
    "chapters/introduction.typ",
    "chapters/typesetting.typ",
    "chapters/conclusion.typ",
  ),
)
```

模板会按照给定顺序 `include` 这些章节文件。

对真正的书稿而言，章节拆分通常比单文件更容易维护，也更适合多人协作。

4.2 交叉引用

Typst 原生支持标签与引用。给这一节添加标签：

4.3 交叉引用

那么正文中就可以引用它：

“关于章节拆分的讨论见 小节 4.3。”

这种方式比手写“见第 X 页”更可靠，因为分页发生变化时引用仍然会自动更新。

事实 Fact

可维护性来自自动更新。

页码、标题编号和交叉引用都应该由排版系统计算，而不是由作者手工维护。

五. 小结

一本真正可维护的技术书籍，通常只需要几类稳定的视觉元素：

1. 清晰的标题层级；
2. 舒适且一致的正文排版；
3. 少量真正有语义价值的强调；
4. 能够容纳代码、公式、表格和脚注；
5. 自动维护的编号、引用和页眉页脚。

模板负责把这些规则固定下来，而正文只需要描述内容。

反思 Reflection

模板的价值不在于“展示它能做多少效果”，而在于作者可以忘记排版规则，专注于内容本身。

